

Lecture 6 — Eigenvalues and Eigenvectors

18.085 Computational Science and Engineering I • MW 9:30–11, 6-120 • Monday, September 28, 2026

After this lecture you should be able to

- State the eigenvalue equation $\mathbf{Ax} = \lambda\mathbf{x}$ and explain why $\mathbf{x}$ is required to be non-zero.
- Explain why an eigenbasis turns multiplication by a matrix into multiplication by a number.
- Compute eigenvalues by hand from $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ and eigenvectors from the nullspace of $\mathbf{A} - \lambda\mathbf{I}$.
- Say what a defective matrix is, and why the characteristic polynomial is *not* how eigenvalues are computed numerically.

1 What they are

Multiply a vector by a matrix and two things usually happen at once: the vector changes length, and it changes direction. The second one is what makes matrices hard to reason about. So ask: are there any vectors that $\mathbf{A}$ does *not* turn — directions along which $\mathbf{A}$ only stretches?

Definition 1.1 (Eigenvalue and eigenvector). Given any **square** matrix $\mathbf{A}$, an *eigenpair* is a pair $(\lambda, \mathbf{x})$ consisting of a scalar and a **non-zero** vector satisfying

$$\mathbf{Ax} = \lambda\mathbf{x}$$

that is, $\mathbf{A}$ acting on $\mathbf{x}$ gives back a constant multiple of $\mathbf{x}$. We call λ an **eigenvalue** of $\mathbf{A}$, and $\mathbf{x}$ the corresponding **eigenvector**.

Two words in that definition are doing real work.

- **Square.** $\mathbf{Ax}$ and $\mathbf{x}$ have to live in the same space before we can ask whether one is a multiple of the other. A rectangular matrix moves you between two different spaces, so the question does not even parse. (Rectangular matrices get their own version of this story in Lecture 25, the SVD.)
- **Non-zero.** If $\mathbf{x} = \mathbf{0}$ were allowed, then $\mathbf{A}\mathbf{0} = \lambda\mathbf{0}$ holds for *every* λ and every $\mathbf{A}$, and the definition would tell us nothing at all. The content of the equation is that some genuine direction survives $\mathbf{A}$ intact.

2 Why they are important

Eigenvectors simplify the behaviour of a matrix: along an eigenvector, multiplying by a matrix is like multiplying by a number. That is already a large simplification for one vector. The point is that it can be made to hold for *all* vectors at once.

If $\mathbf{A}$ is $n \times n$ and we can find n **linearly independent** eigenvectors $\mathbf{v}_1, \dots, \mathbf{v}_n$, then those eigenvectors form a special basis of $\mathbb{R}^n$ — the **eigenbasis** of $\mathbf{A}$. Being a basis (Lecture 4), it lets us write any vector as

$$\mathbf{x} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \dots + c_n\mathbf{v}_n,$$

and now watch what $\mathbf{A}$ does. By linearity, and then by the eigenvalue equation applied one term at a time,

$$\mathbf{A}\mathbf{x} = c_1\mathbf{A}\mathbf{v}_1 + \cdots + c_n\mathbf{A}\mathbf{v}_n = c_1\lambda_1\mathbf{v}_1 + c_2\lambda_2\mathbf{v}_2 + \cdots + c_n\lambda_n\mathbf{v}_n.$$

The coefficients did not mix. Each one was simply rescaled by its own eigenvalue. In these coordinates $\mathbf{A}$ is *diagonal* — much simpler.

The eigenbasis is the good basis

In an arbitrary basis, $\mathbf{A}\mathbf{x}$ couples all n coordinates together. In the eigenbasis it does not couple them at all: coordinate i is multiplied by λ_i and left alone. Finding eigenvectors is finding the coordinates in which the problem falls apart into n independent scalar problems.

Superposition, again

The computation above is the superposition principle of Lecture 1 in new clothing: decompose, act on the pieces, add. Last lecture we decomposed a load into point sources and added Green's functions. Here we decompose a vector into eigenvectors and add scaled copies. It is the same move; only the basis has changed, and this time the basis was chosen to suit the *operator* rather than the data.

3 How to calculate eigenvalues and eigenvectors

In practice

By numerical algorithms — the QR algorithm, inverse iteration, and their relatives. In MATLAB, Python, or Julia you call `eig()` and its relatives, and someone else's very carefully engineered code does the work. We will build one of the ingredients, the QR factorisation, in Lecture 10.

Formally

Eigenvalues are the solutions of

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0$$

which is a polynomial equation in λ , the *characteristic polynomial*.

Where does that come from? Move everything to one side of the eigenvalue equation:

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x} \iff \mathbf{A}\mathbf{x} - \lambda\mathbf{I}\mathbf{x} = \mathbf{0} \iff (\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0}.$$

We are asking for a *non-zero* $\mathbf{x}$ in the nullspace of $\mathbf{A} - \lambda\mathbf{I}$. A square matrix has a non-trivial nullspace exactly when it is singular (Lecture 4), and a square matrix is singular exactly when its determinant vanishes. Hence $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$. The eigenvalues are precisely the numbers λ that make $\mathbf{A} - \lambda\mathbf{I}$ singular.

Eigenvalues are where a matrix loses rank

An eigenvalue is a number λ for which $\mathbf{A} - \lambda\mathbf{I}$ fails to be invertible. So the hunt splits cleanly in two: first find the λ that make $\mathbf{A} - \lambda\mathbf{I}$ singular, then, for each one, read the eigenvectors off as the nullspace of that singular matrix. The subspace machinery of Lecture 4 is doing the second half of the work.

$\lambda = 0$ is the singular case

Put $\lambda = 0$ into that condition and it reads $\det \mathbf{A} = 0$. So a matrix has 0 as an eigenvalue exactly when it is singular, and the eigenvectors for $\lambda = 0$ are exactly the nullspace vectors of Lecture 4. That is worth carrying with you: the difference between the invertible $\mathbf{K}$ and the singular $\mathbf{B}$ of Lecture 2 — fixed ends against free ends — is the difference between “no zero eigenvalue” and “a zero eigenvalue”. The physical statement *a free-free bar can drift* and the algebraic statement *$\mathbf{B}$ has $\lambda = 0$* are one statement in two languages.

Good for hand calculations only

Solving $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$ is fine for small matrices worked by hand. **Do not use it for numerics of large matrices.** Forming an n th-degree polynomial and then rooting it is a numerically disastrous route to the eigenvalues: the roots of a polynomial can be wildly sensitive to tiny changes in its coefficients, so you would be solving a badly conditioned problem in place of a well conditioned one. Real algorithms never form the characteristic polynomial.

4 A worked example

Take

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}.$$

Step 1: the characteristic polynomial.

$$\det(\mathbf{A} - \lambda \mathbf{I}) = \det \begin{bmatrix} 3 - \lambda & 1 \\ 1 & 3 - \lambda \end{bmatrix} = (3 - \lambda)^2 - 1.$$

Step 2: the eigenvalues. Setting it to zero,

$$(3 - \lambda)^2 - 1 = 0 \implies (3 - \lambda)^2 = 1 \implies 3 - \lambda = \pm 1,$$

so there are two eigenvalues,

$$\lambda_1 = 4, \quad \lambda_2 = 2.$$

Step 3: the eigenvectors. Each eigenvalue was chosen to make $\mathbf{A} - \lambda \mathbf{I}$ singular, so now we read off its nullspace. For $\lambda_1 = 4$,

$$\mathbf{A} - 4\mathbf{I} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies x_1 = x_2,$$

and for $\lambda_2 = 2$,

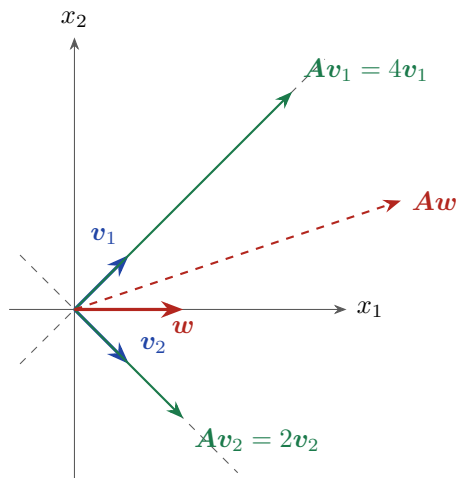
$$\mathbf{A} - 2\mathbf{I} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies x_1 = -x_2.$$

Both matrices have rank 1, so each nullspace is a line, and we pick one vector from each:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Step 4: check. Always check — it costs one multiplication:

$$\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \end{bmatrix} = 4 \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ -2 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$



The two dashed lines are the eigendirections: vectors on them come back scaled, by 4 and by 2. The red vector w is on neither line, and it comes back *turned* — it is a mixture of v_1 and v_2 , and the two pieces were stretched by different amounts.

The same picture, in coordinates

Make that last sentence quantitative. The red vector is $w = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$, and in the eigenbasis it is

$$w = 1 \cdot v_1 + 1 \cdot v_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Now apply the rule from Section 2 — scale each coefficient by its own eigenvalue and add:

$$Aw = 4v_1 + 2v_2 = \begin{bmatrix} 4 \\ 4 \end{bmatrix} + \begin{bmatrix} 2 \\ -2 \end{bmatrix} = \begin{bmatrix} 6 \\ 2 \end{bmatrix},$$

and that is exactly what the matrix gives directly:

$$\begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 6 \\ 2 \end{bmatrix}.$$

Notice what the middle computation did *not* contain: any multiplication by A . Once the vector was written in the eigenbasis, the matrix was replaced by the two numbers 4 and 2. And notice *why* Aw points somewhere new: the v_1 -part was quadrupled while the v_2 -part was only doubled, so the mixture came out with different proportions. A matrix turns vectors because it stretches different eigendirections by different amounts.

Eigenvectors are directions, not vectors

The eigenvectors can be scaled by an arbitrary constant: if $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ then $\mathbf{A}(c\mathbf{v}) = c\mathbf{A}\mathbf{v} = \lambda(c\mathbf{v})$ for any $c \neq 0$. So what an eigenvalue determines is a *line* through the origin, not a particular arrow on it. That freedom is why software returns eigenvectors normalised to unit length, and why your hand answer of $(1, 1)$ and the computer's $(0.7071, 0.7071)$ are the same answer.

5 Defective matrices

Everything in Section 2 assumed we could find n linearly independent eigenvectors. That is the typical case, but it is not guaranteed.

Definition 5.1 (Defective matrix). Generally, it is possible for an $n \times n$ matrix not to have n linearly independent eigenvectors. In this case the matrix is said to be **defective**.

A defective matrix has no eigenbasis, so it cannot be diagonalised, and the clean picture of Section 2 breaks down. You will see an example on the problem set. Defective matrices are not the focus of this course — the matrices we care about come from physical equilibrium problems, are symmetric, and as we will see next time are never defective.

Where this is going

Next time we package this lecture's computation into a single matrix factorisation. Putting the eigenvectors into the columns of a matrix $\mathbf{V}$ and the eigenvalues into a diagonal matrix $\mathbf{\Lambda}$ turns “ $\mathbf{A}$ is diagonal in the eigenbasis” into the statement $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$, the *eigendecomposition*.

Keep one detail of today's example in view: $\mathbf{A}$ was symmetric, and its two eigenvectors $(1, 1)$ and $(1, -1)$ came out perpendicular, with both eigenvalues real. Neither of those was an accident, and both are theorems about symmetric matrices that we prove next lecture.

Reading

Strang, *Computational Science and Engineering*, §1.5.